7 Steps of Highly Effective Problem Solving

If we want to perform well in the interviews we need to **prepare** and play with the **best tactics**.

The 7 Steps

1. Understand the problem
2. Solve the problem manually
3. Come up with a solution idea
4. Play devil's advocate against your idea
5. Do the implementation
6. Comprehensive test cases
7. Clean up your code

1. Understand the Problem

If you understand something **different** than what is asked, then the odds are extremely **low** that you land on a correct solution.

Each sentence of the text **reveals** something about the problem. Sometimes it may not be clear after the first round of reading. In such times, check the **input/output** and its **explanation**.

Don't try to go too fast. Don't try to cut to the chase immediately.

If something is written there, there must be a reason for that.

2. Solve the Problem Manually

‘Nothing can be automated that cannot be done manually!’

Solving a problem programmatically means telling the computer how to solve it via a language it can understand.

Computers can do things at **scale**, a lot **faster** than humans can.

If you don't know how to solve the problem for **small** input, how can you tell a computer to solve it at scale?

Sometimes, solving it by **hand** reveals some patterns that are key to the programmatic solution.

Observe yourself, and understand how you do it by hand to automize.

3. Come up with a solution idea on paper

What comes after solving it by hand?

The big picture and patterns are **not always visible** at first glance.

At this step, it is important to check your algorithm's **time and space complexity**, including the helper functions you use

Rookie Mistake

Skipping time and space complexity checks.

Finally, flesh out the **details** of your algorithm.

Bruteforce & Optimization

Despite being possibly slow, a brute force algorithm is valuable. It's a **starting point** for optimizations, and it helps you wrap your head around the problem.

Techniques for Optimization

Look for **BUD** (**B**ottlenecks, **U**nnecessary Work, **D**uplicated Work):

- Identify bottlenecks in the algorithm that slow down overall runtime.
- Look for unnecessary work and eliminate it where possible.
- Consider duplicated work and find ways to reduce redundant computations.

4. Play devil’s advocate against your idea before falling in love with it

We love to love our ideas.

At the same time, it is better to fall in love with them after getting to know them well.

Therefore, before falling in love, play **devil's advocate** against them.

Imagine you're a critical reviewer or a tester whose job is to thoroughly validate this idea.

Your goal is to find any possible **weaknesses**, **edge cases**, or **scenarios where the idea might fail**.

Ask yourself: How can this idea be **broken**? Can I think of an input or situation that would cause it to **fail**? What are the logical or edge cases that need to be tested?

TIP

A tip that can be useful for coming up with edge cases is going over your proposed solution and looking for assumptions you might have made.

Time is gold, especially in the interviews.

5. The implementation, that is our mission.

Pseudocode is a high-level representation of a solution that **outlines the steps and logic** without adhering to any specific programming language syntax.

pseudocode helps in writing **code faster** and reduces the chances of introducing errors during implementation.

Do not try to explain everything to the interviewer. At the same time, inform them about what is going on after completing each block of code.

Divide your code into **blocks** and using helper functions whenever you can.

1. Choose the best language fit
2. Variable naming
3. Use helper functions
4. Explain code blocks after finishing

6. Comprehensive Test Cases

After implementing the solution, proceed to test with interesting cases, including edge ones, to make sure the solution works for different scenarios.

Testing is not just coming up with **obvious** edge cases like **null nodes**, **empty arrays**, or **zero values**. It is also not trying some random cases.

Each test case you try should have a motivation.

At this stage, do not just think about your algorithm. Imagine the code is written by a 3rd party. Your job is to make sure your tests save the codebase from merging a buggy or inefficient code.

Testing for Assumptions

When developing code or solutions, it is common to make assumptions about certain conditions or expectations. By intentionally testing these assumptions, we can ensure the accuracy and reliability of our codebase.

Let's consider an example of assumption testing. Suppose we have implemented the prefix sum algorithm, **assuming** that all input numbers are **positive**.

To prevent merging a potentially flawed codebase, we must design test cases that verify this assumption. One test case would involve providing an array with **negative** numbers as **input**.

7. Simplify and Clean Up Your Code

Now that we have a working solution and we are confident in its **accuracy** and **efficiency** let's do some **cleanup** and **simplifications**.

Consider better variable naming, adding some comments, and possibly modularizing further. This step can be optional, depending on how much time you have left.

Example: First Unique Character In a String

Given a string s, find the first non-repeating character in it and return its index. If it does not exist, return -1.

Pseudocode for optimal approach:

1. Initialize an empty dictionary
2. Iterate through each char in the input string
    a. If char is already a key in dictionary Increment its value by 1
    b. If char is not in the dictionary, add char as a key with a value of 1
3. Iterate through the input string again
    a. If frequency of char is equal to 1 Return char
4. If no non-repeated character is found, return -1

```python
class Solution:
    def firstUniqChar(self, s: str) -> int:
        freq_dict = defaultdict(int)
        for char in s:
            freq_dict[char] += 1
        for i in range(len(s)):
            if freq_dict[s[i]] == 1:
                return i
        return -1
```

Quote of the Day

“Practice doesn't make perfect, perfect practice makes perfect”

Vince Lombardi